

딥러닝에서 XOR 문제

- 딥러닝 → 뉴럴 네트워크에서 히든 레이어를 2개 이상 확장
 - 레이어에 각 노드를 몇 개 넣을 건지 자유
 - 활성화함수 조건 : 비선형, 미분가능
- 여러 hidden layer → deep learning
 - XOR: 단층 → OK ; 딥러닝 → NO 왜?
 - = vanishing gradient

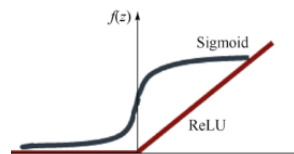
```
actFunc = 'sigmoid'
model = tf.keras.Sequential()
model.add(layers.Dense(10, activation=actFunc, input_dim=2))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(1, activation='sigmoid'))
```

Gradient Vanishing Problem

- XOR을 단층 네트워크 → OK ; 다층 네트워크(딥러닝)으로 확장 → NO 왜?
 - Backpropagation 과정에서 sigmoid의 미분값을 계산.
시그모이드의 입력($WX+b$)의 절대값이 크면 기울기가 거의 0. 가중치 업데이트 X
 - 이 식에서 $h(1-h) = S(x)(1-S(x))$ 의 $W_i = W_i - t(h_{i-1})\lambda\delta_i$, $\delta_i = \delta_{i+1}t(W_{i+1})h_i(1-h_i)$
최댓값 : $h=1/2 \rightarrow 1/4$;
많은 레이어 통과 → δ_i 에 0.25 보다 작은 수 계속 곱함 → 결국 0이 됨

ReLU

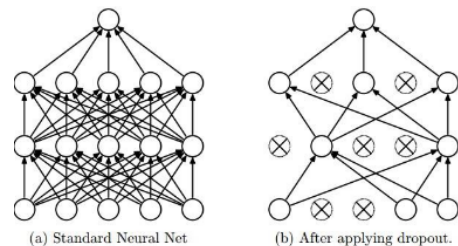
- 솔루션 : 중간 Hidden layer에 시그모이드 대신 ReLU($x = \max(0, x)$)를 사용.
이 함수는 연속적, 미분가능(0을 기점으로 0,1의 기울기)
 - 시그모이드처럼 음수를 0, 양수는 bypass처리;
0으로 처리된 노드는 학습 X.
→ leaky ReLU, maxout을 사용
- 코드에서 마지막 layer = sigmoid?
 - 시그모이드를 거치면 0~1 사이 확률로 출력



```
actFunc = 'relu'
model = tf.keras.Sequential()
model.add(layers.Dense(10, activation=actFunc, input_dim=2))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(10, activation=actFunc))
model.add(layers.Dense(1, activation='sigmoid'))
```

Xavier Initialization & Drop out

- Xavier 초기값 : 초기값 설정에 따라 수렴 속도 차이
 - 가중치의 분산을 일정수준 이하로 작게 → 더 빨리 수렴함
 - $\text{np.random.randn}()$: 표준정규분포상의 난수, fan_in(out) : 입력(출력)노드 수
 - He initialization : xavier의 개선이라 주장
 $W = \text{np.random.randn}(fan_in, fan_out) / \text{np.sqrt}(fan_in)$ # Xavier initialization(2010)
 $W = \text{np.random.randn}(fan_in, fan_out) / \text{np.sqrt}(fan_in/2)$ # He initialization(2015)
- Drop out : 레이어마다 어떤 노드를 죽일지 랜덤 결정
 - Fully connected neural net : forward할 때 랜덤으로 노드 죽임. 왜?
 - 몇 노드들이 가중치가 너무 크면 나머지 노드가 있으나 마나
 - 나머지 노드들도 학습 시키기 위해



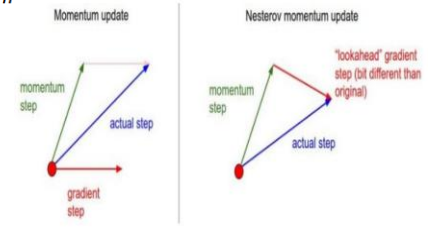
- 딥러닝 → 뉴럴에서 히든을 2개 ↑ 확장. 활성화함수 조건 : 비선형, 미가
- Gradient vanishing problem : Xor : 단층O, 다층(딥러닝)X ? → BP중 sigmoid의 기울기 $wx+b \uparrow \rightarrow$ 기울기 ≈ 0 . 업데이트 $h(1-h)$ 의 최댓값 : $h=1/2 \rightarrow 1/4$; 수많은 레이어 통과 → δ_i 에 $1/4$ 보다 작은 수 계속 곱함, 언더플로우. 결국 기울기 0
- ReLU : GV의 솔루션 : sigmoid 대신 $\text{ReLU}(x) = \max(0, x)$, 비선형&미가 → 0처리 노드 학습 X → Leaky ReLU, maxout
- Xavier initialization : (초기값 → 수렴속도 차이) → 가중치 분산 일정수준 ↓ → 더 빨리 수렴
- Drop out : forward시 레이어마다 랜덤 노드 죽임. 왜? 몇 노드 가중치 ↑ ↑ → 나머지노드 학습 X, 나머지 노드도 학습 위해

Batch Normalization

- 딥러닝 중간 레이어의 출력 정규화 ; 이전 레이어의 출력을 정규화해 학습
→ 초기값에 상관X. $N(\beta, \gamma^2)$ 을 가지는 분포; 학습할수록 감마 베타의 오차가 작아짐
 $BN(x_i) = \gamma \cdot \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$
`model.add(tf.keras.layers.BatchNormalization())`

Optimizer : 학습중 모델의 가중치 업데이트 방식 결정; 성능 최적화

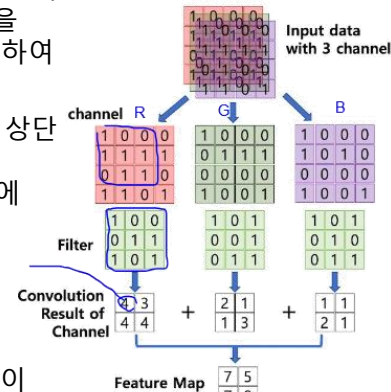
- 필요성 : SGD중 비효율적 학습경로 개선(업데이트 경로 과도하게 흔들림 ↓), 더 빠르고 효율적 해 수렴. 성능 ↑
- Momentum : 모멘텀(관성)처럼 이전 방향이 현재 방향에 영향을 줌. $m \leftarrow \alpha \cdot m + \lambda \frac{\partial Cost(w)}{\partial w}$
 $w \leftarrow w - m$
 $\alpha \cdot m$ = 알파만큼 이전 모멘텀 반영
- Nesterov Momentum : 미래에 한스텝 움직일 예상 위치에서 경사 계산, 미래 경향성 고려 $m \leftarrow \beta m + \lambda \frac{\partial Cost(W - \beta m)}{\partial w}$, $w \leftarrow w - m$
- 학습률 개선:
손실함수가 작아져서 개선
→ 파라미터 갱신
손실함수 갱신X → 학습률/2
 $w_1 = w_0 - \lambda \frac{\partial Cost(w)}{\partial w}$
if $L(w_1) < L(w_0)$:
 $w_0 = w_1$
else:
- AdaGrad :
Gradient의 제곱의 누적값 h를 $lr = lr/2$
파라미터마다 계산 → 벡터형태로 저장.
그레디언트의 크기가 클수록 학습률이 작아짐. 조기종료하는 경향. $h \leftarrow h + \frac{\partial L}{\partial w} \odot \frac{\partial L}{\partial w}$
 $w \leftarrow w - \eta \frac{1}{\sqrt{h}} \frac{\partial L}{\partial w}$
- RMSProp : AdaGrad의 단점 보완. 변화량의 제곱 누적 X. 지수 평균 $G \leftarrow \gamma G + (1 - \gamma)(\nabla_{\theta} J(\theta_t))^2$
→ 최근 변화량이 중요 반영.
- Adam optimizer : 모멘텀, RMSProp 방식 결합. 가장 널리/성능 ↑. Momentum에 의해 최적 경로로 가는 경로의 진동을 방해. 파라미터 벡터가 해로 가는 속도를 비슷하게 맞춰 해를 빠르게 수렴.
 $\theta = \theta - \frac{\eta}{\sqrt{G + \epsilon}} \cdot \nabla_{\theta} J(\theta_t)$



- Batch normalization : 딥러닝 중간 레이어 정규화 & 이전 레이어 출력 정규화 학습 → 초기값에 상관X; $X \sim N(\beta, \gamma^2)$ 를 가지는 분포 $BN(x_i) = \gamma \cdot \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$; 학습할수록 γ, β 오차 ↓.
- Optimizer : 모델의 가중치 업데이트 방식 결정, 성능 최적화. 왜? SGD중 비효율적 학습경로 개선(업데이트 경로가 과도하게 흔들림 ↓), 더 빠르고 효율적 수렴. 성능 ↑.
 - Momentum : 이전방향 → 현재 방향에 영향. $m \leftarrow \alpha \cdot m + \lambda \frac{\partial Cost(w)}{\partial w}$ $w \leftarrow w - m$, α 만큼 반영
 - Nesterov momentum : 미래 한스텝 움직일 예상 위치에서 경사 계산, 미래 경향성 고려 $m \leftarrow \beta m + \lambda \frac{\partial Cost(W - \beta m)}{\partial w}$, $w \leftarrow w - m$
 - 학습률 개선 : loss ↓ 개선 → 파라미터 갱신, 아니면? (학습률)*1/2 $w_1 = w_0 - \lambda \frac{\partial Cost(w)}{\partial w}$ if $L(w_1) < L(w_0)$: $w_0 = w_1$ else: $lr = lr/2$
 - Adam : 모멘텀+RMSProp, momentum에 의해 경로 진동 ↓, 파라미터 벡터가 해로 가는 속도 비슷하게. 빠른 수렴

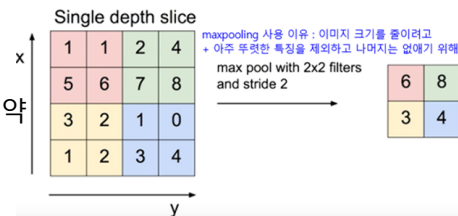
Convolutional layer

- 이미지를 픽셀 단위로 직접 분류 문제에 사용 → 비효율적;
Convolution filter = 자동 특징 추출기(Feature extractor). 이미지의 원본 픽셀을 직접 사용하는 대신, 이미지 픽셀의 특징으로 바꾸어 사용; CNN → 자동으로 특징 추출
- CNN 처리 과정
 - 1) 이미지의 Feature를 Extraction
 - 2) Feature를 이용해 딥러닝 수행
 → 단순화된 특징 : 이미지 구별O인 간결한 특징만. 판별도움X 정보는 삭제
- Convolutional filter의 작동 방식 : 이미지에 '컨볼루션 필터'라는 작은 행렬 겹침 → 이미지와 필터의 위치 픽셀값을 모두 곱한뒤 더해 하나의 숫자 생성 → 이 과정을 반복하여 feature map이라는 새로운 이미지 생성
 - 이미지 : 4*4 크기의 R,G,B 채널, 필터 : 3*3 ;> 왼쪽 상단에서 필터, 채널 elementwise곱 ;하나의 값 완성.
3채널에 반복. → 필터를 우로 한칸 → ... → 우하단에 필터 도착? 멈춤 (elementwise product = 각자리에 대응되는 원소값을 단순히 곱하고 더함)
- Stride : 이미지 N*N, 필터 F*F, 이동회수(stride) S → 필터 통과후 이미지 크기 = $\frac{N-F}{S} + 1$; 통과? → 작아짐
이미지 줄어듦 → 판단에 사용 숫자 줄어듦 → 연산 용이
- Zero padding : 원본 이미지에 0으로 이루어진 부분 추가; 이미지 크게 만듦. (N=5*5, F=3*3) → N'=3*3; N에 zero padding; 7*7 → N'=5*5; 이미지 크기가 유지되도록; + '0'을 포함해서 어느 부분이 가장자리인지 알려줄 수 있기 때문
- 여러장의 feature map 생성 : 1개 이미지
 - 여러개 필터로 처리, 다양한 feature map 생성. 각 필터의 숫자는 난수 지정
 - 학습과정에서 backpropagation으로 수정 (convex 하다면 항상 loss 감소)
 - 이유? : 1장 feature map에서 놓칠 수 있는 특징을 다른 feature map에서 포착 가능



Pooling layer

- Convolution layer가 있다면? → pooling layer O
convolution layer의 값중 대표값을 하나 뽑아 요약 (최대, 평균 등... 일반적으로 max pooling)
** 2*2 커널을 2*2 stride(우)



- Convolution filter=Feature extractor : 이미지 원본 픽셀X, 이미지 픽셀의 특징으로 바꿈. CNN→자동 특징 추출
 - CNN 처리 과정 : 1) Feature extract, 2) Feature→딥러닝, 구별O인 간결 특징만. 도움X정보X→단순화된 특징
 - Convolution filter를 이미지 겹침→(필터값)*(픽셀값)으로 곱하고 더해서 새 숫자 → 이 과정을 반복, 'Feature map' 새 이미지 생성
 - Stride : $\left(\frac{N-F}{S} + 1\right)$, 이미지→판단 사용 숫자 ↓, 연산 용이
 - Zero padding : 원본 이미지 가장자리, 0인 부분 추가→ Feature map크기 유지 + 0을 통해 어디가 가장자리?
 - 여러 Feature map 생성 : 각 필터 난수 지정→학습중 BP로 수정. 왜? 1장 FM에서 놓칠 수 있음 다른 FM에서
- Pooling layer : CL?→Pooling layer O, CL값중 대표값 → 요약(일반적 : maxpooling)

Mnist data에서 적용

```

- model = models.Sequential()
  model.add(layers.Conv2D(16, (5,5), activation='relu', input_shape=(28, 28, 1)))
  model.add(layers.MaxPooling2D((2, 2)))
  model.add(layers.Conv2D(32, (3, 3), activation='relu'))
  model.add(layers.MaxPooling2D((2, 2)))
  model.add(layers.Flatten())
  model.add(layers.Dense(10, activation='softmax'))
  model.summary()

```

```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 16)	416
max_pooling2d (MaxPooling2D)	(None, 12, 12, 16)	0
conv2d_1 (Conv2D)	(None, 10, 10, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 32)	0
flatten (Flatten)	(None, 800)	0
dense (Dense)	(None, 10)	8,010

Total params: 13,066 (51.04 KB)
 Trainable params: 13,066 (51.04 KB)
 Non-trainable params: 0 (0.00 B)

- 입력 레이어 : 28*28*1 (1은 컬러X, 단색이미지란 뜻)
- 1번째 합성곱 레이어 :
 - 5*5 합성곱 필터, 16개의 feature map 생성, Relu 활성화 함수 사용. Stride = 1;
 - 파라미터수 : $416 = \{5*5(\text{필터크기}) * 1(\text{입력채널}) + 1(\text{바이어스})\} * 16(\text{필터 수})$
 - 이미지 크기 : $28*28*1 \rightarrow 24*24*16$
- 1번째 max pooling 레이어 :
 - 2*2 크기 풀링 필터, stride = 2; $24*24*16 \rightarrow 12*12*16$ 로 줄어듦
 - 맥스풀링 레이어 → 파라미터 필요 X. 왜? 4개 숫자중 가장 큰거 하나만 선택 ok
- 2번째 합성곱 레이어 : 3*3 필터, 32개 feature map 생성, Relu;
 - 파라미터 수 = $(3*3*16 + 1) * 32 = 4640$
 - 이미지 크기 : $12*12*16 \rightarrow 10*10*32$
- 2번째 max pooling : 2*2 필터, stride = 2; $10*10*32 \rightarrow 5*5*32$; 파라미터 = 0
- Flatten layer: Pooling 에서 5*5*32 feature map → 1개의 벡터로 펼침 : $5*5*32 = 800$ 개 숫자; 파라미터 X
- Dense layer : 800개 벡터 입력 → 10개의 뉴런으로 반환;
 - 10개 뉴런 → 0~9까지 10개의 클래스;
 - 출력층엔 softmax를 활용해 각 클래스에 속할 확률 계산
 - 파라미터 수 = $800(\text{입력 특징}) * 10(\text{출력 뉴런}) + 10(\text{bias}) = 8010$
- 총 파라미터 수 = $416 + 4640 + 8010 = 13066$ 개 학습

- 1번째 CL : 5*5 CF, 16개 FM → param # = 416 = [(필터크기)*(채널)+(bias)]*(필터수); 크기: 28*28*1 → 24*24*16
 - 1번째 MP: 24*24*16 → 12*12*16, param # = 0; ∴ 4개중 큰 거 1개만
 - 2번째 CL : 3*3 CF, 32 FM → param # = 4640 = [(3*3)*16+1]*16; 크기: 12*12*16 → 10*10*32
 - 2번째 MP: 10*10*32 → 5*5*32, param # = 0
 - Flatten layer : 5*5*32를 하나의 벡터로, 5*5*32 → 1*800
 - Dense layer : softmax : 1*800 → (입력 Feature)*(출력뉴런)+(bias)=800*10+10=8010
- 입력 layer : 28*28*1(컬러채널, 흑백)

이미지 파일 → 행렬로 바꾸는 법, ImageDataGenerator

- ```
base_dir = 'd:/Cat_Dog_smallData'
train_dir = os.path.join(base_dir, 'train')
img = image.load_img(train_dir + '/dogs/dog.0.jpg', target_size=(28,28))
img = image.img_to_array(img)
print(img)
```
- 왜? 컴퓨터는 이미지를 그림X, 숫자로 표현.  
+ 모델학습을 위해선 행렬/벡터로 입력 해야함
- ```
train_generator = train_datagen.flow_from_directory(train_dir,
target_size=(150, 150), batch_size=20, class_mode='binary')
validation_generator = test_datagen.flow_from_directory(validation_dir,
target_size=(150, 150), batch_size=20, class_mode='binary')
```
- 크기에 상관없이 150*150; 1500*1500? GPU 감당X
15*15? 잘 못맞춤
150*150 & 배치사이즈 줄임 OK

배치 정규화와 규제(kernel Regularization) $\min(\|Y - X(\theta)\|_2^2 + \lambda \|\theta\|_2^2)$

- 배치 정규화 : 미니 배치 처리시 배치의 입력 데이터의 분포가 달라짐 → 학습할 때 해가 불안정하게 수렴 → 분포의 평균, 분산을 일정한 값으로 정규화
- Kernel regularization : 레이어를 쌓을시 생기는 오버피팅을 방지;
let 회귀방정식 모수 = 세타(베타와 동일) → 세타의 제곱 더함 → 파라미터 수 ↓
→ 복잡하게 만들수록 람다항 커짐; 세타항을 너무 쓰지 않고도 loss 최소화;
→ 파라미터 수가 늘어날수록 더 잘맞춘다 → data set을 아예 외워버린다(=오버피팅)
→ 해결 : 노드를 끈다(베타를 0으로) 어떤 노드를 끄는지? 학습으로 → 오버피팅을 방지하고 모델을 더 깊게 쌓을 수 있음

ImageNet challenge

- VGGNet, GoogleNet, ResNet은 Transfer Learning에 주로 사용되는 Pre-trained CNN
→ Pre-trained : 미리 트레이닝 된 네트워크; 이미 공개된 버전 → 약간 커스터마이징

Transfer Learning

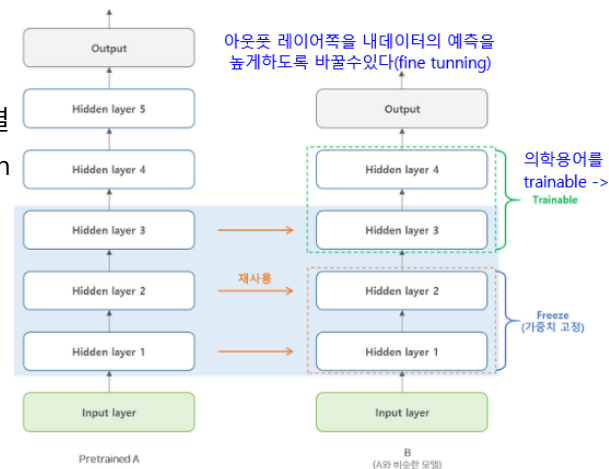
- 전이학습 : Pre-trained network를 계승해 해당 모델과 유사 문제 해결
→ CNN : Input layer → Convolution layer → Fully Connected layer → Output layer
Convolution layer만 재사용.
FC layer를 새로 구성 → convolution layer 학습 필요 리소스 절약

→ VGG16 사용 예 :

Include_top : FC_layer 사용X
conv_base.trainable :

(학습 → VGG16 파라미터 바꿈?) → 전이학습에선 X

- VGG 모델 자체가 완벽 → conv O, FC X, feature selection만 사용
안좋은 컴, data로 못맞출수도



```
conv_base = VGG16(weights='imagenet', include_top=False, input_shape=(150, 150, 3))
# 기존 VGG 16 모델을 가져오겠다 ↑
model = models.Sequential()
model.add(conv_base) # 파라미터를 가져오고
model.add(layers.Flatten()) # 플래튼
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
conv_base.trainable = False
```

Cat_Dog_VGG_Train_Tf2.py

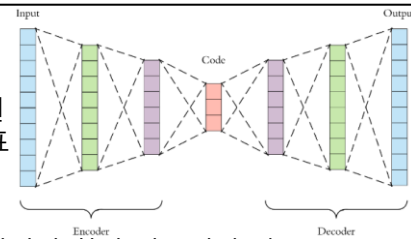
- 이미지→행렬 : 왜? 컴퓨터는 이미지를 그림X, 숫자로 표현.+ 모델학습을 위해선 행렬/벡터로 입력 해야함
- 배치 정규화 : 미니배치→배치의 입력분포 달라짐→학습시 해가 불안정 수렴→ 분포의 평균, 분산을 정규화
- Kernel regularization $\min(\|Y - X(\theta)\|_2^2 + \lambda \|\theta\|_2^2)$; 학습 데이터를 잘 맞추나 아예 외움: 오버피팅 → λ항이 커짐
→ θ항을 많이 사용X & cost 최소화가 목표, 어떤θ를 지우는지는 θ=0으로 만들고 BP로 수정
- Pre-trained network : 이미 트레이닝된 네트워크, 공개된 버전 → 커스터마이징
- 전이학습 : pre trained network를 계승. 해당 모델과 유사문제 해결. Convolution layer는 재사용, Fully connected layer만 fine tuning

Auto Encoder

✓

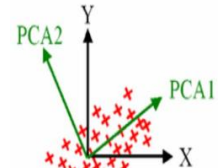
오토인코더

- 정의 : 입력데이터 → 저차원 코드로 압축(=인코더)
+ 이 코드를 다시 원본 데이터와 같은 차원으로 복원(Decoder); 최대한 (입력데이터 = 출력데이터)가 목표
- Input이 히든레이어를 통과했다가 재현.
암호화&복호화. PCA의 비선형버전
(활성함수는 무조건 비선형. 선형으로 답하게 쌓아봐야 하나 쌓는것과 같음)



PCA(Principal Component Analysis)

- 다차원 변수의 상관관계를 사용해 차원을 축소; X = 복싱, Y = 마라톤 → PCA1 = 유산소 능력(공통요인), PCA2 = 그 외 실력에 영향을 주는 능력(고유영역)
- PCA1만으로 충분히 많은 정보? → 불필요한 차원을 줄이기 가능(2D → 1D)
- 정규화가 되었음을 가정,
X,Y 분포를 합치면 2
→ PCA1,PCA2의 분포를 합치면 2.
PCA1의 분포 > 1이면
PCA2의 분포 < 1
→ 설명에 필요한 PCA1만 남기고 PCA2는 지워버림
→ 불필요 차원 감소



$$X \sim (\mu, \Sigma) \rightarrow \text{자기들끼리 상관됨.}$$

$$\rightarrow \text{독립 } \Sigma = \Gamma \cdot D_{\lambda} \cdot \Gamma'$$

→고유벡터

Γ is an p by p orthonormal matrix composed of p eigenvectors

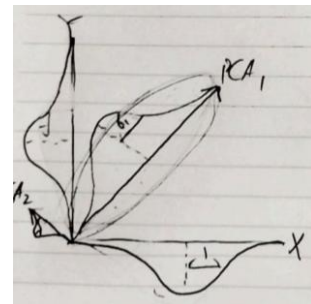
D_{λ} is an p by p diagonal matrix composed of p eigenvalues

$$PCA' = \Gamma \cdot X' \quad \text{분산=information의 양} \uparrow$$

$$y = AX \quad \rightarrow \quad \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$$

$$X \in \mathbb{R}^{(n,2)} \Rightarrow \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^{(n,2)} \quad \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \quad \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} & \dots & \sigma_{1n} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} & \dots & \sigma_{2n} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 & \dots & \sigma_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \sigma_{n3} & \dots & \sigma_n^2 \end{pmatrix}$$

$X \sim N(\mu, \Sigma)$ if Σ 가 대칭행렬, 양정부호행렬 then $\Sigma = \Gamma \cdot D_{\lambda} \cdot \Gamma^T$
→ $PCA' = \Gamma \cdot X'$ 가 성립
 Γ : eigen vector (주성분), D_{λ} : eigen value (분산, information의 양)



- PCA 예제 :

```
# x1,x2,x3가 서로 연관되게.
x = np.array([[1, 2, 3],
              [2, 4, 5],
              [3, 7, 6],
              [4, 7, 7]])

def standardScaler(X): # 표준화
    return ((X - np.mean(X, axis = 0))
            / np.std(X, ddof = 1, axis = 0))

x_norm = standardScaler(x)
np.cov(x_norm, ddof=1, rowvar=0)
# 표준화된 데이터의 공분산 = 상관계수와 같아진다
from sklearn.decomposition import PCA
pca = PCA(n_components=3)
proj = pca.fit_transform(x_norm) # PCA Score
proj # 결과가 뭐냐면 4개 -> 3개로 바뀜.
# 아래것이 PCA1, PCA2축이고 X1,X2,X3는 서로 독립적
diag = np.diag(pca.explained_variance_) # eigen values
diag
# 3개를 더하면 3이 나온다. 내가 한축만 쓰고 버린다면?
# -> 정보손실 = 1-2.9/3 = 0.03333

array([[2.92512342, 0.        , 0.        ],
       [0.        , 0.05805056, 0.        ],
       [0.        , 0.        , 0.01682602]])

eVector = pca.components_.T
cov = np.dot(np.dot(eVector, diag), eVector.T); cov
# 감마 * D감마 * 감마'

array([[1.        , 0.9486833 , 0.98270763],
       [0.9486833 , 1.        , 0.95618289],
       [0.98270763, 0.95618289, 1.        ]])
```

- AE : 입력→저차원 코드 압축(=인코더) → 코드를 원본과 같은 차원으로 복원(=디코더); 최대한 입력=출력 목표 PCA의 비선형 버전, 활성함수는 비선형 왜? 선형이면 하나만 쌓는것과 동일
- PCA : 변수간 상관관계 → 차원축소. 공통/고유 영역에서 공통
→ 많은 설명? 고유 지움;
정규화를 가정, (X분포)=(Y분포)=1, 합=2 → (PCA1 분포)>1?, (PCA2 분포) <1 → PCA2 X. 불필 차원 ↓

$$y = AX \quad \rightarrow \quad \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$$

$$X \in \mathbb{R}^{(n,2)} \Rightarrow \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^{(n,2)} \quad \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \quad \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} & \dots & \sigma_{1n} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} & \dots & \sigma_{2n} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 & \dots & \sigma_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \sigma_{n3} & \dots & \sigma_n^2 \end{pmatrix}$$

$X \sim N(\mu, \Sigma)$ if Σ 가 대칭행렬, 양정부호행렬 then $\Sigma = \Gamma \cdot D_{\lambda} \cdot \Gamma^T$
→ $PCA' = \Gamma \cdot X'$ 가 성립
 Γ : eigen vector (주성분), D_{λ} : eigen value (분산, information의 양)

Auto Encoder(계속)

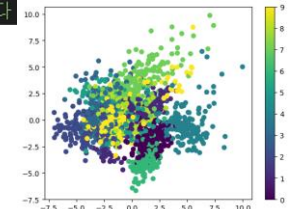


PCA - 8*8 숫자 이미지를 3차원으로 축소 예제

```
print(digits.data.shape) # digits = (1797, 3) 데이터로 8*8 숫자 이미지다.
fig = plt.figure(figsize=(6, 6)) # figure size in inches
for i in range(64):
    ax = fig.add_subplot(8, 8, i + 1, xticks=[], yticks=[])
    ax.imshow(digits.images[i], cmap=plt.cm.binary, interpolation='nearest')
from sklearn.decomposition import PCA
pca = PCA(n_components=64)
proj = pca.fit_transform(digits.data) # PCA Score
proj.shape # proj는 PCA를 통해 (1797,3) 데이터로 64개의 차원을 3차원으로 줄인 데이터다.
diag = np.diag(pca.explained_variance_); diag

array([[7.34477606e+00, 0.00000000e+00, 0.00000000e+00, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [0.00000000e+00, 5.83549054e+00, 0.00000000e+00, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [0.00000000e+00, 0.00000000e+00, 5.15396118e+00, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       ...,
       [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00]])

k = 17
np.sum(diag[:k,:k]) / 64 # 71퍼센트만 쓸 수 있다 -> 64개를 17개로.
# 26%만 데이터를 썼는데 71퍼센트; 파일 크기가 21퍼센트로 압축했는데 71퍼센트를 유지
# 주성분의 크기를 다 합치면 64; 첫번째 주성분이 7.34개의 성분을 설명할 수 있다
eVector = pca.components_
# 3차원으로 줄였어도 3차원 공간에서 점들이 군집화 되어 있어
# 0~9까지의 숫자의 특징을 어느정도 보유하고 있음을 알 수 있다.
plt.scatter(proj[:, 0], proj[:, 1], c=digits.target)
plt.colorbar() # 비슷한 모양끼리 군집을 이룸
```



Noise Reduction

- AE : 주성분 변환시 선형변환으로? 비선형이면서 잘설명 안되나?
- Noise Reduction → 차원축소시 뚜렷하지 않은 특징 제거.
노이즈를 제거해야 latent layer 학습 가능
→ Flatten과 convolution 층을 사용한 AE → 노이즈 제거



Anomaly detection

- AE는 정상신호를 복원. 비정상 신호를 복원? → 학습한적X, 복원률↓.(정상 신호(7→7) 간에는 KL distance가 작음. 근데 비정상 신호(A→8)간엔 KLd 큼)

Colorization

- AE를 이용해 그레이 이미지 → 컬러 이미지. 흑백 = 입력 → 컬러 = 출력 으로 학습 후 새로운 흑백? → 컬러. 단, 컬러복원X, 새칠하기

Image Search

- AE의 좌측을 학습해 code의 가중값 확보. Decode를 제거한 모형으로 다음을 수행
 - 1) 이미지 DB의 이미지를 Encoded 벡터로 만들
 - 2) 사용자의 이미지를 Encoded 벡터로 만들
 - 3) kNN 기법으로 유사도 측정. 가장 가까운 이미지?

- Noise Reduction : 차원 축소시 뚜렷X 특징 제거, 노이즈X어야 latent layer 학습O. flatten+convolution AE → noise reduction
- Anomaly detection : AE : 비정상 복?->학습X, 복원률 ↓, KLd ↑ ↑
- Colorization : 흑백→컬러 순으로 학습. 단, 컬러복원X. 색칠하기
- Image search : AE incoder → code O, → 1) (DB 이미지 → Encoded 벡터) 2) (사용자 이미지→encoded 벡터) 3) KNN 기법. 유사도 측정. 가까운 이미지?

RNN(Recurrent Neural Network)

Recurrent Neural Network

- Time Series Forecasting : 시간 흐름에 따라 순서O 데이터를 기반, 미래 예측; AR(autoregression model) : t시점 값은 t-1시점 값(자기자신)의 영향을 받음. 주가예측/자연어 처리등 시간이 중요한 분야 사용

$$Y_t = \phi Y_{t-1} + \epsilon_t$$

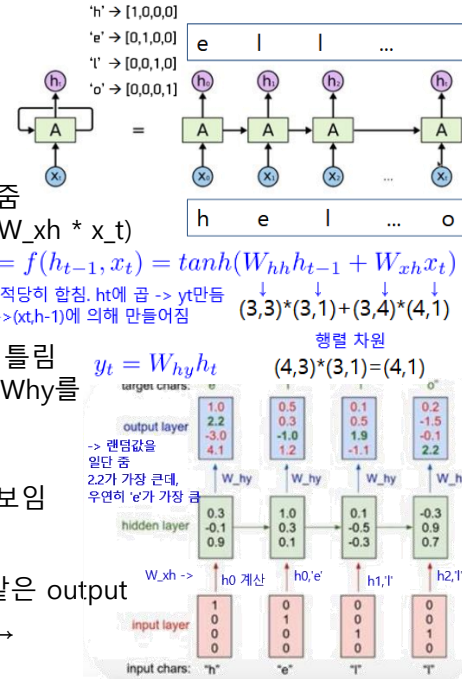
모수 : 모음 -> 0으로 예측

$$Y_t = \phi Y_{t-1} + \theta \epsilon_{t-1} + \epsilon_t$$
- RNN 정의 : 과거의 정보(ht-1)가 현재상태 결정에 영향을 주는 구조. 순환 : 과거의 자기값이 현재로 다시 피드백
 - HT = F(HT-1, XT) 형태에서, F에는 tanh같은 활성화함수 사용. Whh, Wxh 모두 학습 가능파라미터
 - Yt는 모든 과거데이터(Y1...Yt-1)에 영향 O. 하지만 Yt-1 ... Y1 순으로 영향받음
- 여러 RNN 모델 :
 - One to one : 한 입력→한 출력
'love'→'사랑'; 일대일 번역
 - One to many : 한 입력 → 여러 출력
이미지→해수욕장,가족,non-adult, ...
 - Many to one : 여러 입력 → 한 출력 글/음성 내용 분석 → 하나로 요약
 - Many to many : 여러입력 → 여러입력; '안','녕'→'하','세','요'; 광고카피 등



Char RNN

- 목표 : hello 학습 : h→e, he→l, hel→l, hell→o
- One-hot-encoding : 모델 학습? → 각 문자를 수치벡터로 치환
 - 'h' → [1,0,0,0]
 - 'e' → [0,1,0,0]
 - 'l' → [0,0,1,0]
 - 'o' → [0,0,0,1]
- 학습 과정 : 1) 먼저 h0값의 파라미터를 난수로 줌
2) 다음 ht(hidden layer) = tanh(W_hh*h_t-1 + W_xh * x_t)
3) h_t를 구했다면 W_hy*h_t = y_t를 구함 $h_t = f(h_{t-1}, x_t) = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$
4) 그럼 yt를 이용해 yt+1을 구하고 ... $y_t = W_{hy}h_t$
5) 그 다음 각 초록색 값 들이 가장 커야 하는데 틀림
→ loss 발생, backpropagation으로 Whh, Wxh, Why를 재조정 후 다시 학습
- Prediction 모델과 Generative AI의 차이점
 - Prediction : 학습한걸 대답; 같은 input? → 같은 output
 - 컷독 : 뭘 줘도 컷/독으로, 연봉 : (연봉리스트 → 다음 연봉?) → 10명의 평균
 - Generative : 학습한 대답X; 학습 분포→새로운걸 생성; 같은 input?→다른 output
 - 컷독 : 컷/독 아닐수도?; 연봉 : 이런 조건의 연봉도 있음 → 동일값을 줘도 다른값
- TimeDistributed()
 - RNN 출력시 모든 output을 반영, many to many 출력. (앞들? 중요, 뒤들? 중요)
 - ↔ many to one 방식 : 마지막 output 결과만 중요 (앞들? X, 뒤들? 중요)



- Time series Forecasting : 시간O순서 데이터→미래예측 // AR : t시점 값→ t-1시점 값에 영향
- RNN : 과거정보(h_(t-1))→ 현재상태 결정 영향. $h_t = f(h_{t-1}, x_t) = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$; $Y_{t-1}...Y_0$ 순
- Char CNN 과정 : (OHE)→h_0난수→(h_t→y_t=W_hy*h_t)→ ... → 각 output layer의 loss를 BP로 조정
- Prediction vs Generative : prediction : 학습한 대답. 같I→같O, GAI : 학 대답X, 분포→새 생성. 같I→다O
- TimeDistributed() : RNN 출→ 모든 Output 반영. Many to many ↔ many to one : 마지막 output만

Many to one RNN

- 입력시 발생하는 문제 : g,o,o,d → 1; b,a,d → 0, w,o,r,s,e → 0, s,o ,g,o,o,d → 1;
문제점? x_data의 길이가 다르다. → padding 사용
→ X_data가 긴게 들어올 수 [[6, 7, 7, 4], [3, 2, 4], [10, 7, 8, 9, 5], [9, 7, 1, 6, 7, 7, 4]]
있으니 일단 10칸을 비워둠 → 나중에 one hot encoding. 단, 6774000000에서 6 자체도 one hot encoding으로 10차원 벡터. 즉, 6774000000은 10 * 10 짜리 행렬
- 축소 임베딩 : one hot encoding의 차원을 줄여
차원의 저주(=입력 차원 ↑ ↑, 계산 효율성/성능 ↓) 해결.
→ 적은 차원으로 단어 표현
+ 비슷한 의미 문장 임베딩 → 둘 간 유사도 1에
가까움 (one-hot-encoding : 비슷한 단어들
서로 다른 취급 -(임베딩) > 유사도 1에 가까워짐), 임베딩 벡터 간 코사인 유사도가 높
기 때문. 모델이 단어 간 의미적 관계/유사성 인지 OK
- 구현방법 : (CBOW)특정단어→주변단어 예측,(skip-gram)주변단어→특정단어 예측
(오토인코더 기반) AE 모델 중간의 프로젝션 벡터(잠재공간)를 임베딩 벡터로;
ex) AE : 개 → 개; 늑대 → 늑대; 늑대, 개가 비슷한 projection layer.
→ 유사단어들이 인베딩 공간상 가까운 위치;

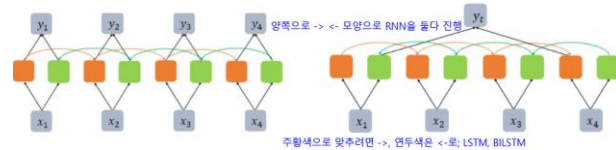
	good	bad	worse	so good
	[6, 7, 7, 4]	[3, 2, 4]	[10, 7, 8, 9, 5]	[9, 7, 1, 6, 7, 7, 4]
one hot encoding	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]
	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]	[0 0 0 0 0 0 0 0 0 0]

the =>	0	0	0	0	1
cat =>	1	0	0	0	0
sat =>	0	0	0	1	0

cat =>	1.2	-0.1	0.3	3.2
mat =>	0.4	2.8	-0.9	0.6
on =>	2.1	0.3	0.1	0.4

Long Short Term Memory(LSTM, 장단기 기억 신경망)

- 정의 : LSTM은 Rnn의 고질적 문제인 장기 의존성 해결 위해 고안.
필요성 : RNN은 직전시점의 정보에 큰 영향. But, 먼 과거 정보가 결정적 영향?
Ex) I grew up in France, I speak fluent [French] → French를 맞추기 위함?
France > fluent; fluent등 덜 중요? → 덜 입력. France 처럼 많이 중요? → 많이 입력
→ Forget gate, input gate : 이전→다음 state
넘어갈때 적절히 forget; 너무 많은 hidden layer
방지 + 과거 중요 정보 선택적으로 가중 ↑ → RNN 더 깊게 가능
- BiLSTM : 시퀀스를 정방향,
역방향으로 처리하는 LSTM 2개
사용 → 더 풍부한 문맥 정보
기반 예측



- Many to one RNN : 입력시 발생 문제 : x_data 길이 다 다름→padding. 나중에 OHE; 6774000000→10*10 행렬
- 축소 임베딩 : ohe의 차원 ↓, 차원의 저주(=입력차원 ↑ ↑→계산효율/성능 ↓)해결→적은차원으로 단어표현
+ 비슷한 의미 문장/단어 임베딩 → 둘 간 유사도≈1 (ohe:(비슷?→다름)-(임베딩)→유사도≈1)→단어 간 의미/관계
- 구현방법:(CBOW)특정단어-(예측)→주변단어, (skip-gram)주단→특단, AE:code=임베딩벡터→유사? 임공가값
- LSTM:(RNN문제:장단기 의존성 해결 위함), (먼과거→결정적 영향?)→덜중?덜입. 많중?많입.
- Forget gate, input gate : 이전→다음 state시 적절 forget. Hidden layer ↑ ↑ X. 과거중요정보가중→RNN 깊
- BiLSTM : 시퀀스를 정방향, 역방향으로 처리 LSTM 2개. 더 풍부 문맥정보 기반 예측.

총정리

딥러닝 → 뉴럴에서 히든을 2개 ↑ 확장. 활성화함수 조건 : 비선형, 미가

- Gradient vanishing problem : Xor : 단층O, 다층(딥러닝)X ? → BP중 sigmoid 기울기 $w x + b \uparrow \rightarrow$ 기울기 ≈ 0 . 업데이트 $h(1-h)$ 의 최대값 : $h = \frac{1}{2} \rightarrow \frac{1}{4}$: 수많은 레이어 통과 → δ_i 에 $\frac{1}{4}$ 보다 작은 수 계속 곱함, 언더플로우. ∴ 기울기 0
- ReLU : GV 솔루션 : sigmoid 대신 $\text{ReLU}(x) = \max(0, x)$, 비선형 & 미가 → 0처리 노드 학습 X → Leaky ReLU, maxout
- Xavier initialization : (초기값 → 수렴속도 차이) → 가중치 분산 일정수준 ↓ → 더 빨리 수렴
- Drop out : forward시 레이어당 랜덤 노드 죽. 왜? 몇 노드 가중치 ↑ ↑ → 나머지노드 학습 X, 나.노 학습 위해
- Batch normalization : 딥러닝 중간 레이어 정규화 & 이전 레이어 출력 정규화 학습 → 초기값에 상관X; $X \sim N(\mu, \sigma^2)$ 를 가지는 분포 $BN(x_i) = \gamma \cdot \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$; 학습할수록 γ, β 오차 ↓.
- Optimizer : 모델의 가중치 업데이트 방식 결정, 성능 최적화. 왜? SGD중 비효율적 학습경로 개선(업데이트 경로가 과도하게 흔들림 ↓), 더 빠르고 효율적 수렴. 성능 ↑.
 - Momentum : 이전방향 → 현재 방향에 영향. $m \leftarrow \alpha \cdot m + \lambda \frac{\partial \text{Cost}(w)}{\partial w}$ $w \leftarrow w - m$ α 만큼 반영
 - Nesterov momentum : 미래 한스텝 움직일 예상 위치서 경사 계산, 미래 경향성 고려 $m \leftarrow \beta m + \lambda \frac{\partial \text{Cost}(w - \beta m)}{\partial w}$, $w \leftarrow w - m$
 - 학습률 개선 : loss ↓ 개선 → 파라갱신, 아니면? (학습률) $^{1/2}$ $w_1 = w_0 - \lambda \frac{\partial \text{Cost}(w)}{\partial w}$ if $L(w_1) < L(w_0)$: else: $w_0 = w_1$ $lr = lr/2$
 - Adam : 모멘텀+RMSProp, momentum에 의해 경로 진동 ↓, 파라 벡터가 해로 가는 속도 비슷. 빠른 수렴

CNN

- Convolution filter=Feature extractor : 이미지 원본 픽셀X, 특징으로 바꿈. CNN → 자동 특징 추출
- CNN 처리 과정 : 1) Feature extract, 2) Feature → 딥러닝, 구별O인간결 특징만. 도움X정보X → 단순화된 특징
- Convolution filter를 이미지 겹침 → (필터값)*(픽셀값)으로 곱하고 더해서 새 숫자 → 이 과정을 반복, 'Feature map' 새 이미지 생성 - Stride : $\left(\frac{N-F}{S} + 1\right)$, 이미지 → 판단 사용 숫자 ↓, 연산 용이
- Zero padding : 원본 이미지 가장자리 0인 부분 추가 → Feature map크기 유지 + '0' → 어디가 가장자리?
- 여러 Feature map 생성 : 각 필터 난수 지정 → 학습중 BP로 수정. 왜? 1장 FM에서 놓칠 수 있음 다른 FM에서
- Pooling layer : CL? → Pooling layer O, CL값중 대표값 → 요약(일반적 : maxpooling)

CNN - Mnist data

- 1번째 CL : 5*5 CF, 16개 FM → param# = 416 = [(필터크기)*(채널)+(bias)]*(필터수); 크기: 28*28*1 → 24*24*16
- 1번째 MP: 24*24*16 → 12*12*16, param # = 0; ∴ 4개중 큰 거 1개만
- 2번째 CL : 3*3 CF, 32 FM → param # = 4640 = [(3*3)*16+1]*16; 크기: 12*12*16 → 10*10*32
- 2번째 MP: 10*10*32 → 5*5*32, param # = 0
- Flatten layer : 5*5*32를 하나의 벡터로, 5*5*32 → 1*800
- Dense layer : softmax : 1*800 → (입력 Feature)*(출력뉴런)+(bias) = 800*10+10 = 8010

CIFAR10/ImageNet challenge/Transfer Learning

- 이미지 → 행렬 : 왜? 컴퓨터는 이미지를 그림X, 숫자로 표현. + 모델학습을 위해선 행렬/벡터로 입력 해야함
- 배치 정규화 : 미니배치 → 배치의 입력분포 달라짐 → 학습시 해가 불안정 수렴 → 분포의 평균, 분산을 정규화
- Kernel regularization : $\min(\|Y - X(\theta)\|_2^2 + \lambda \|\theta\|_2^2)$; 학습 데이터를 잘 맞추나 아예 외움: 오버피팅 → λ 항이 커짐 → θ 항을 많이 사용X & cost 최소화가 목표, 어떤 θ 를 지우는지는 $\theta = 0$ 으로 만들고 BP로 수정
- Pre-trained network : 이미 트레이닝된 네트워크, 공개된 버전 → 커스터마이징
- 전이학습 : pre trained network를 계승. 해당 모델과 유사문제 해결. Convolution layer는 재사용, Fully connected layer만 fine tuning

총정리(계속)

AE : 입력→저차원 코드 압축(=인코더) → 코드를 원본과 같은 차원 로 복원(=디코더);

- 최대한 입력=출력 목표.PCA의 비선형 버전, 활성화함수는 비선형 왜? 선형이면 하나만 쏘는것과 동일
- PCA : 변수간 상관관계 → 차원축소. 공통/고유 영역에서 공통
→ 많은 설명? 고유 지움;
정규화를 가정, (X분포)=(Y분포)=1, 합=2 → (PCA1 분포)>1?,
(PCA2 분포) <1 → PCA2 X. 불필 차원 ↓
- Noise Reduction : 차원 축소시 뚜렷X 특징 제거, 노이즈X어야 latent layer 학습O. flatten+convolution AE → noise reduction
- Anomaly detection : AE : 비정상 복?->학습X, 복원률 ↓, KLd ↑ ↑
- Colorization : 흑백→컬러 순으로 학습. 단, 컬러복원X. 색칠하기
- Image search : AE incoder → code O, → 1) (DB 이미지 → Encoded 벡터) 2) (사용자 이미지→encoded 벡터)
3) KNN 기법. 유사도 측정. 가까운 이미지?

Handwritten notes showing the PCA derivation:

$$y = AX + b \rightarrow \begin{pmatrix} y \\ 1 \end{pmatrix} = \begin{pmatrix} X \\ 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}$$

$$X \in \mathbb{R}^{n \times d} \Rightarrow \begin{pmatrix} X \\ 1 \end{pmatrix} \in \mathbb{R}^{(n+1) \times d}$$

$$\begin{pmatrix} X \\ 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} a_1 & a_2 & \dots & a_d \\ b \end{pmatrix}$$

$$X \sim N(\mu, \Sigma) \quad \text{if } \Sigma \text{ is diagonal, } \Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2) \text{ then } \Sigma = \Gamma \cdot D \cdot \Gamma^T$$

$$\rightarrow \text{PCA} = \Gamma \cdot X'$$

Below the equations, it says: Γ : Eigen vector (기대값), D : Eigen value (고유, singular value)

RNN : 과거정보(h_(t-1))→ 현재상태(h_t) 결정 영향

- Time series Forecasting : 시간O순서 데이터→미래예측 // AR : t시점 값→ t-1시점 값에 영향
- $h_t = f(h_{t-1}, x_t) = \tanh(W_{hh} * h_{t-1} + W_{hx} * x_t)$; $Y_{t-1} \dots Y_0$ 순
- Char CNN 과정 : (OHE)→h_0난수→(h_t→y_t=W_hy*h_t)→ ... → 각 output layer의 loss를 BP로 조정
- Prediction vs Generative : prediction : 학습한 대답. 같I→같O, GAI : 학 대답X, 분포→새 생성. 같I→다O
- TimeDistributed() : RNN 출→ 모든 Output 반영. Many to many ↔ many to one : 마지막 output만
- Many to one RNN : 입력시 발생 문제 : x_data 길이 다름→padding. 나중에 OHE; 6774000000→10*10 행렬
- 축소 임베딩 : ohe의 차원 ↓, 차원의 저주(=입력차원 ↑ → 계산효율/성능 ↓) 해결→적은차원으로 단어표현 + 비슷 의미 문장/단어 임베딩 → 둘간 유사도≈1 (ohe:(비슷?→다름)-(임베딩)→유사도≈1)→단어간 의미/관계
→ 구현:(CBOW)특정단어-(예측)→주변단어, (skip-gram)주단→특단, AE:code=임베딩벡터→유사? 임공간값
- LSTM:(RNN문제:장단기 의존성 해결 위함), (먼과거→결정적 영향?)→덜중?덜입. 많중?많입.
→ Forget gate, input gate : 이전→다음 state시 적절 forget. Hidden layer ↑ ↑ X. 과거중요정보가중→RNN 깊
- BiLSTM : 시퀀스를 정방향, 역방향으로 처리 LSTM 2개. 더 풍부 문맥정보 기반 예측.